

Oficina Docker:

Aula 1:

Como era o mundo tech antes da virtualização? Como era pra um jovem dev na faculdade de ti UFSM ter um ambiente geral (seu pc) e ter que instalar várias coisas diferentes. Usar Windows mas precisar de Linux pra testar o código pra Pitthan (ter que usar dual boot ou outro pc, muito trabalho).

Virtualização ajuda com isolamento de ambientes e aí fica mais fácil configurar tudo isso.

Então VMs resolvem todos os problemas? Pois bem vocês me digam: não! Porque ainda tem que configurar toda essa bomba e é uma mão desgraçada.

Além disso, VMs são pesadas.

Como funciona uma VM? (puxar a [img que o dinei usou as 14min do video dele](#) e explicar ela (redesenhar a imagem com um tom light no tablet da monique))

Um pc tem os recursos pra rodar 1 (fazer piada com pc ruim), talvez 2 sistemas operacionais simultaneamente (VMs). Ao chegar em 3 e principalmente além disso, é bem provável que o computador comece a travar bastante, puramente porque é muito intensivo no consumo de recursos

O que é o Docker? (explicar o que é docker) Docker é virtualização a nível de sistema operacional (não roda máquinas virtuais, cada uma com seu próprio OS) chamados containers. O kernel linux contido no chamado motor Docker (Docker engine) é “emprestado” e reutilizado pelos containers. Por esse motivo, eles são muito mais leves, rápidos (no uso e principalmente no boot) e versáteis.

(comparar a imagem mostrada antes de como funcionam VMs em camadas com o OS com a versão atualizada para containers (redesenhar tb a outra, talvez fazer uma transição riscando parte da primeira?) lembrar de explicar que caso o usuário use Windows como sistema operacional o docker engine roda uma VM de kernel linux *vamos entrar com mais detalhes nisso logo logo (WSL 2)* (apesar de ser uma VM ela foi EXTREMAMENTE otimizado pela microsoft para se tornar enxuta e rodar de forma integrada com o windows* (caso o usuario habilite WSL 2))

Instalação: passar pelo walktrough da instalação, mostrar imagens do docker desktop, explicar que a aula vai ser mais focada nos comandos por cli porque daí os botões no docker desktop ficam bem auto intuitivos e explicativos, minha intenção é vocês entenderem o funcionamento por trás da coisa, sem usar atalhos inicialmente.

Como funciona Docker: explicar o funcionamento de containers e imagens Docker, explicando também a sintaxe de Imagem:Tag e as diferentes tags que podem ter (lembrar latest e discussões sobre). Uma mesma imagem pode ser referenciada por várias tags, e podem ser definidas várias tags para uma mesma imagem, pois cada uma será uma imagem

diferente. Além disso, nada impede localmente você rodar diferentes containers com a mesma imagem. Ligar essa explicação com:

Docker hub: apresentar o docker hub, explicar a parte de trusted content e mostrar algumas imagens como exemplo. Explicar o que é o dockerfile (receita de bolo, manual de instruções que o Docker usa para criar uma imagem) e por isso mesmo imagens que não são “trusted content” no geral são seguras pois é tudo código aberto, então tende a ser tranquilo rastrear o como aquela imagem foi criada (e alterar se assim o usuário desejar)

Aula 2:

Comandos para trabalhar com containers Docker:

- docker container run <imagem>

Cria e executa um novo container da imagem especificada

- docker container ls

Lista os containers em execução, use -a para listar containers parados também

- docker container exec

Executa um comando em um container em execução

- docker container cp

Copia um arquivo entre o container e a máquina hospedeira (host)

- docker container start / stop / kill / rm

Inicia / pára / "mata" / exclui um container já criado

Rodar container do docker/getting-started:

<<https://hub.docker.com/r/docker/getting-started>>

Rodar manualmente na hora com a tag :vscode

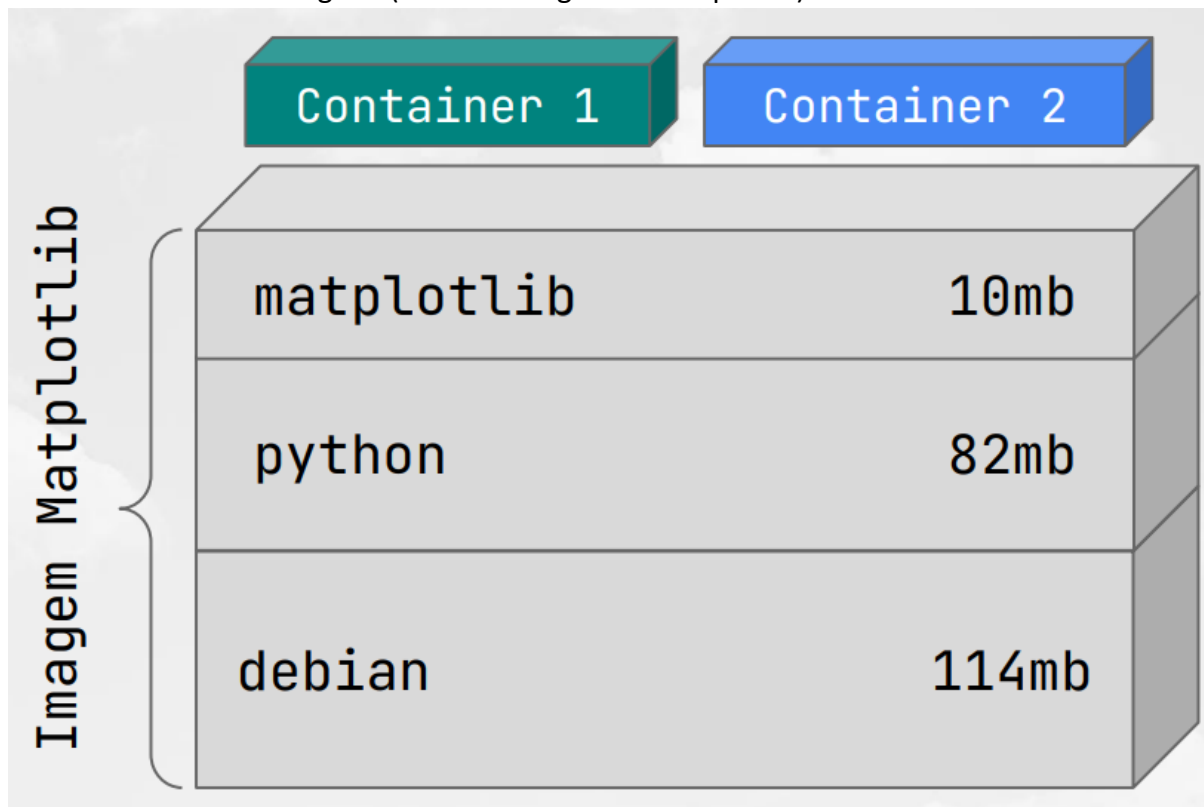
Aplicação prática com Flask:

Lembrar de alterar o site, notando que as mudanças vão ser aplicadas porque o container tá mapeado pra puxar os arquivos do diretório da minha máquina*

Comentar sobre como ainda foi muito trabalhoso e demorou muito, e podemos acelerar esse processo criando uma imagem personalizada:

Como funcionam imagens Docker?

Entrar na explicação sobre camadas. Um container é apenas uma “camada alterável” sobre as camadas fixas da imagem. (mostrar imagem do matplotlib)



Em outro slide mostrar duas imagens (image layers docker 1 e 2 (<https://docs.docker.com/get-started/docker-concepts/building-images/understanding-image-layers/>)) explicando sobre como as camadas podem ser reutilizadas para containers/situações diferentes

ATIVIDADE PARA ENTREGAR SOBRE A AULA 2

Aula 3

Comandos para trabalhar com imagens Docker:

· docker image ls

Lista as imagens no repositório local

· docker image build <pasta> -t <tag>

Constrói uma imagem a partir de um Dockerfile

· docker image rm <imagem>

Remove uma imagem do repositório local

· docker image pull <imagem>

Baixa uma imagem de um repositório remoto (é implícito no run)

· docker image push <imagem>

Envia uma imagem construída localmente para um repositório remoto

Mencionar que ainda tem a parte de docker compose que vai ser vista posteriormente

Fazer o exemplo prático de flask dessa vez criando uma imagem com o Dockerfile

Lembrar de alterar o site, dessa vez notando que nada vai mudar pois os arquivos estão agora dentro do container rodando (pode entrar no container e alterar lá pra mostrar as mudanças)*

ATIVIDADE PARA ENTREGAR SOBRE A AULA 3

Aula 4:

Docker compose:

"Docker Compose é uma ferramenta para definir e rodar aplicações Docker multi-contêineres. [...] com um único comando, você cria e inicia todos os serviços presentes no seu arquivo de configuração."

O docker compose na teoria pode ser usado pra múltiplos containers, mas geralmente é usado para orquestrar vários (até porque a maior parte das aplicações vai usar mais de um container). Na virtualização com máquinas virtuais se eu tivesse que usar MySQL, Python e R, eu subiria minha VM e instalaria tudo isso dentro de uma mesma VM, pois era inviável subir uma VM e instalar Python, subir outra e instalar MySQL, etc. Com Docker isso não é só possível como muito fácil: é mais simples pegar a imagem do Python já pronta no Dockerhub, do MySQL já pronta no Dockerhub e subir vários containers, um com cada coisa.

Ele automaticamente cria uma rede que interliga e facilita a comunicação entre todos os containers. (explicar isso melhor)

Docker compose não passa de um arquivo YAML (.yml) com a configuração dos containers da aplicação. Exemplo:

services:

web:

image: php:8.0-apache

banco:

image: mysql:8.0

Comandos para trabalhar com Docker Compose:

· docker compose up

Constrói, (re)cria e inicia os serviços. Use -d para iniciar em segundo plano.

Use -- build para forçar a compilação da imagem

. docker compose down

Pára e destrói os recursos (exceto volumes permanentes) criados pelo comando up

· docker compose stop

Pára os serviços iniciados pelo comando up -d

Aplicação prática usando Flask+Redis

Lembrar de acessar o redis-cli com os containers rodando pra poder mexer na chave hits.

ATIVIDADE PARA ENTREGAR SOBRE A AULA 4